

A COOKBOOK FOR NODE.JS DEVELOPERS

Temporal Durable Workflows Cookbook

Build workflows that survive crashes, retries and restarts — in Python, framed for a developer who already knows queues, async/await and n8n.

Durable execution

Workflows

Activities

Retries

Signals

Schedules

Testing

Versioning

AI workflows

mini-n8n

Written for engineers fluent in Node.js / TypeScript who want long-running, reliable business processes without hand-rolling state machines.

Python SDK first, with the TypeScript equivalent beside it — every pattern answers: what it solves, how you would do it with a queue, how Temporal does it, and when NOT to use it.

Edition 2026 · Python SDK (temporalio) · TypeScript notes · build-your-own n8n capstone

Table of Contents

How to Use This Book	4
The one idea: your code keeps running, even when your servers don't.	4
What this book covers	4
How each chapter is built	5
Setup assumed	5
Part 1 • Why Temporal (Durable Execution)	6
1.1 The problem, in one example	6
1.2 How it works (the 60-second version)	6
1.3 Temporal next to what you know	7
1.4 The vocabulary	8
1.5 When NOT to use Temporal	8
Part 2 • Running Temporal Locally	9
2.1 The dev server (one command)	9
2.2 Running it in Docker	9
2.3 A Python project	10
2.4 The Web UI tour	10
2.5 The CLI you'll actually use	11
Part 3 • Core Concepts: Workflows, Activities, Workers	12
3.1 Hello, durable world	12
3.2 Who runs what	14
3.3 Prove the durability to yourself	14
3.4 Data in and out	14
Part 4 • Activities: Where Side Effects Live	16
4.1 What belongs in an activity	16
4.2 Timeouts: always set one	16
4.3 Retries are on by default	16
4.4 Idempotency: the rule that makes retries safe	17
4.5 Long activities: heartbeats	18
4.6 Sync activities and blocking libraries	18
Part 5 • Workflow Rules: Determinism	20
5.1 Why the rules exist	20
5.2 The rules, as a table	20
5.3 The Python sandbox	21
5.4 Logging and printing	21
5.5 Changing code of running workflows	21
Part 6 • Talking to Running Workflows	23
6.1 An approval workflow	23
6.2 Updates: a signal that answers back	24
6.3 Timers	25
6.4 Cancellation	25
Part 7 • Composition: Parallelism, Children, Sagas	27
7.1 Run activities in parallel	27
7.2 Child workflows	27
7.3 Continue-as-new: workflows that run forever	28
7.4 Sagas: undo on failure	28
Part 8 • Starting Workflows: IDs, Dedup and Schedules	30
8.1 Start vs execute	30

8.2 Workflow IDs are your dedup key	30
8.3 Schedules: cron, but durable	31
8.4 Starting from Node	32
Part 9 • Testing Workflows	33
9.1 The time-skipping test environment	33
9.2 Mock activities	33
9.3 Signals in tests	34
9.4 Replay tests: catch non-determinism before deploy	34
9.5 Test activities as plain functions	34
Part 10 • Versioning and Production	36
10.1 Changing a workflow that's already running	36
10.2 Workers in production	36
10.3 Data: types and privacy	37
10.4 Self-host vs Temporal Cloud	37
10.5 Observability	37
Part 11 • AI Workflows on Temporal	39
11.1 The LLM call as an activity	39
11.2 A document pipeline	40
11.3 A durable agent loop	40
Part 12 • Capstone: Build Your Own mini-n8n	42
12.1 The architecture	42
12.2 The definition format	43
12.3 Nodes are plain functions	43
12.4 The durable interpreter	45
12.5 Worker and webhook trigger	46
12.6 A flow to run	48
Part 13 • Cheat Sheet	50
13.1 The skeleton	50
13.2 Workflow-safe replacements	50
13.3 Client calls	51
13.4 Choosing a timeout	51
13.5 Node.js → Temporal map	51
13.6 The five rules	51
13.7 Temporal vs Prefect vs n8n: pick in 10 seconds	52

How to Use This Book

The one idea: your code keeps running, even when your servers don't

Temporal is a **durable execution** platform. You write an ordinary `async` function. Temporal records every step it takes. If the process crashes halfway through, a new worker picks it up **at the exact line where it stopped**, with its local variables intact. Retries, timeouts, waiting for three days and resuming after a deploy are all handled for you.

If you have ever built this by hand, you know the usual pieces: a BullMQ queue, a `status` column in Postgres, a cron job that sweeps stuck rows, and a lot of `if (job.attemptsMade > 3)`. Temporal replaces all of that with code that reads top to bottom.

NODE.JS MENTAL MODEL

Think of a Temporal workflow as an `async` function whose `await`s are **checkpoints**. Every `await` on an activity, timer or signal is saved on the server. A crash doesn't lose the promise chain; the chain gets replayed on another machine and continues from there.

What this book covers

Part	You learn	You can then
1	Why durable execution exists	Tell when Temporal beats a queue, cron, n8n or Prefect
2	Running Temporal locally	Start a dev server + UI in one command
3	Workflows, activities, workers, task queues	Ship a hello-world that survives <code>kill -9</code>
4	Activities, timeouts, retries, heartbeats	Call flaky APIs safely
5	Determinism rules	Avoid the #1 source of Temporal bugs
6	Signals, queries, updates, timers	Build human-in-the-loop approvals
7	Child workflows, parallelism, sagas	Compose big processes, roll back cleanly
8	Workflow IDs and schedules	Replace cron and deduplicate runs
9	Testing and replay	Test a 30-day workflow in milliseconds
10	Versioning and production	Deploy changes without breaking running workflows
11	AI workflows	Durable LLM pipelines and agents
12	Capstone: mini-n8n	Your own JSON-defined workflow engine on Temporal
13	Cheat sheet	One-page recall

How each chapter is built

Every pattern answers four questions: **what problem it solves**, **how you'd do it with a queue in Node**, **how Temporal does it**, and **when not to reach for it**. Look for these callouts:

NODE.JS MENTAL MODEL

The mapping to the Node.js and TypeScript tools you already know, including the Temporal TypeScript SDK.

TIP

A shortcut or habit worth keeping.

COMMON MISTAKE

The trap most people fall into the first time.

WHEN NOT TO USE IT

When Temporal, or this particular feature, is the wrong tool.

EXERCISE

A small build to do before moving on. The sprint plan tracks these.

Setup assumed

You know Python `async/await` (the Python cookbook covers it), you can run Docker, and you have used a job queue before. All code uses the **Python SDK** (`temporalio`) with Python 3.11+ and `uv`. The TypeScript SDK is shown where the mapping helps.

TIP

SDK APIs move fast. When a signature here differs from what your editor shows, trust the installed SDK and the official docs at docs.temporal.io. The concepts in this book stay the same.

Part 1 · Why Temporal (Durable Execution)

1.1 The problem, in one example

An order process: charge the card, reserve stock, email the customer, wait 7 days, ask for a review. Here it is in plain Node:

TYPESCRIPT

```
async function processOrder(order: Order) {
  await payments.charge(order);           // what if the process dies right after this?
  await inventory.reserve(order);         // what if this API is down for 10 minutes?
  await email.sendConfirmation(order);
  await sleep(7 * 24 * 3600 * 1000);      // nobody runs a process for 7 days
  await email.askForReview(order);
}
```

Each line hides a failure mode. A crash after `charge` means the customer paid but never got stock reserved. An outage means you need retries with backoff. The 7-day sleep is impossible in a normal process. So in real life you split this into a queue job per step, a `status` column, a scheduler for the delay and a sweeper for stuck orders. The business logic ends up spread across five files.

With Temporal, the code stays almost exactly like the version above, and it becomes correct:

PYTHON

```
@workflow.defn
class ProcessOrder:
    @workflow.run
    async def run(self, order: Order) -> None:
        opts = dict(start_to_close_timeout=timedelta(seconds=30))
        await workflow.execute_activity(charge, order, **opts)
        await workflow.execute_activity(reserve_stock, order, **opts) # retried
        await workflow.execute_activity(send_confirmation, order, **opts)
        await asyncio.sleep(timedelta(days=7).total_seconds())      # a durable timer
        await workflow.execute_activity(ask_for_review, order, **opts)
```

If the worker dies during the 7-day sleep, nothing is lost. The timer lives on the Temporal server, and any worker picks the workflow up when it fires.

1.2 How it works (the 60-second version)

1. Your **workflow** code runs on **your** worker process, not on the Temporal server.

2. Every time the workflow does something that matters (runs an activity, starts a timer, receives a signal), the worker tells the server and the server appends an **event** to that workflow's **history**.
3. If the worker crashes, another worker downloads the history and **replays** the workflow function from the start. Steps that already happened aren't re-executed; their recorded results are fed back in. The function fast-forwards to where it stopped.
4. That's why workflow code must be **deterministic** (Part 5): replay must take the same path every time.

NODE.JS MENTAL MODEL

It's like event sourcing for a function call. The event log is the source of truth, and the function's in-memory state is rebuilt by replaying the log. Redux devtools "time travel" is a fair picture of it.

1.3 Temporal next to what you know

Need	BullMQ / Celery	n8n	Prefect	Temporal
Run one background job	Great	OK	OK	Overkill
Multi-step process with state	Manual (DB + status)	Visual, limited	Good for data pipelines	Built for it
Survive crash mid-step	Job retries from the start	Execution fails	Retry task (cache helps)	Resumes at exact step
Wait days or months	Delayed jobs, fragile	Wait node	Pause / schedule	Durable timers, free
Human approval mid-flow	Build it yourself	Wait node + webhook	<code>pause_flow_run</code>	Signals / updates
Who writes it	Developers	Anyone	Python developers	Developers
Built-in integrations	None	Hundreds	Some	None — you write activities

TIP

The one-line summary: **n8n** is fastest to a working automation, **Prefect** is the nicest way to run Python data and ML pipelines, and **Temporal** is what you reach for when a process must never lose its place, like payments, orders, onboarding, provisioning, or long AI agents.

1.4 The vocabulary

Term	Meaning
Workflow	Your durable, deterministic orchestration function
Activity	A normal function that does I/O (HTTP, DB, email, LLM). Can fail and be retried
Worker	Your process that polls the server and runs workflows and activities
Task queue	Named channel that routes work to workers (like a BullMQ queue name)
Client	Starts workflows, sends signals, reads results
Event history	The append-only log of everything a workflow did
Workflow ID	Your business key (<code>order-123</code>), unique among running workflows
Namespace	Isolation unit, like a database schema (<code>default</code> locally)

1.5 When NOT to use Temporal

WHEN NOT TO USE IT

Skip it for fire-and-forget jobs (send one email, resize one image), since a queue is simpler. Skip it for pure data pipelines that are mostly "run these Python functions on a schedule and show me the logs", where Prefect is more ergonomic. And skip it when non-developers need to build the flows, which is n8n's job. Temporal also adds a cluster to operate, or a Temporal Cloud bill. Pay that cost when **correctness of a long-running process** is the actual problem.

EXERCISE

Pick one process from your own work (checkout, onboarding, a report that emails a PDF). List each step and write down what happens today if the server dies between steps 2 and 3. Keep the list; you'll rebuild it on Temporal by Part 7.

Part 2 · Running Temporal Locally

2.1 The dev server (one command)

The Temporal CLI ships a complete single-binary dev server with the Web UI built in.

SHELL

```
brew install temporal          # macOS; Linux/Windows: see docs.temporal.io/cli
temporal server start-dev      # gRPC on localhost:7233, UI on http://localhost:8233
```

That's the whole cluster for development: server, persistence and UI in one process. By default it keeps data **in memory**, so a restart wipes history. To keep it between restarts:

SHELL

```
temporal server start-dev --db-filename temporal.db
```

COMMON MISTAKE

Confusing the two ports. **7233** is gRPC, which your Python client and workers connect to. **8233** is the Web UI you open in a browser. `Client.connect("localhost:8233")` hangs or fails with a confusing error.

2.2 Running it in Docker

For a setup closer to production (Postgres persistence, separate UI container), use the official compose files:

SHELL

```
git clone https://github.com/temporalio/docker-compose.git temporal-compose
cd temporal-compose
docker compose up -d          # server on 7233, UI on http://localhost:8080
```

That repo has variants for Postgres, MySQL and Elasticsearch. Pick one and pin its image tags in your own copy so upgrades are deliberate.

NODE.JS MENTAL MODEL

It's the same shape as running Postgres or Redis for a Node app: a stateful service in Docker that your app talks to over a port. The difference is what's stored. Temporal keeps **workflow histories**, and your workers stay stateless.

2.3 A Python project

SHELL

```
uv init temporal-lab && cd temporal-lab
uv add temporalio
```

A layout that scales:

TEXT

```
temporal-lab/
├─ activities.py      # I/O lives here
├─ workflows.py      # deterministic orchestration only
├─ worker.py         # registers both, polls a task queue
└─ starter.py        # client: starts workflows, sends signals
```

TIP

Keep workflows and activities in **separate modules** from day one. The Python SDK runs workflow code in a **sandbox** that re-imports the workflow module. Heavy imports (HTTP clients, ORMs, LLM SDKs) belong in activities, where the sandbox never touches them.

2.4 The Web UI tour

Open <http://localhost:8233> and find these four things. Later chapters refer back to them.

- **Workflows list:** every run with its ID, type, status (Running, Completed, Failed, Timed out) and start time.
- **Event history:** click a run to see every event. `ActivityTaskScheduled`, `ActivityTaskCompleted`, `TimerStarted`, `WorkflowExecutionSignaled`. This is the log that replay uses.
- **Pending activities:** which activity is stuck, which attempt it's on, and the last failure message.
- **Actions:** signal, cancel, terminate, reset. **Reset** re-runs a workflow from an earlier event, which is useful after fixing a bug.

2.5 The CLI you'll actually use

SHELL

```
temporal workflow list
temporal workflow describe --workflow-id order-123
temporal workflow show     --workflow-id order-123      # full event history
temporal workflow start    --type ProcessOrder --task-queue orders \
                           --workflow-id order-123 --input '{"id": "123"}'
temporal workflow signal   --workflow-id order-123 --name approve --input '"ok"'
temporal workflow cancel   --workflow-id order-123      # graceful: workflow can clean
up
temporal workflow terminate --workflow-id order-123     # hard stop, no cleanup
```

EXERCISE

Start the dev server with `--db-filename`, open the UI, and run `temporal workflow list`. It should be empty. You'll fill it in Part 3.

Part 3 · Core Concepts: Workflows, Activities, Workers

3.1 Hello, durable world

Three files. First the activity, a normal function that is allowed to do I/O:

PYTHON

```
# activities.py
from temporalio import activity

@activity.defn
async def greet(name: str) -> str:
    activity.logger.info("greeting %s", name)
    return f"Hello, {name}!"
```

Next the workflow, which orchestrates activities and does no I/O of its own:

PYTHON

```
# workflows.py
from datetime import timedelta
from temporalio import workflow

with workflow.unsafe.imports_passed_through():
    from activities import greet      # imported normally, not re-imported by the
    sandbox

@workflow.defn
class HelloWorldWorkflow:
    @workflow.run
    async def run(self, name: str) -> str:
        return await workflow.execute_activity(
            greet, name, start_to_close_timeout=timedelta(seconds=10)
        )
```

Then the worker and a starter:

PYTHON

```
# worker.py
import asyncio
from temporalio.client import Client
from temporalio.worker import Worker
from activities import greet
from workflows import HelloWorldWorkflow

async def main():
    client = await Client.connect("localhost:7233")
    worker = Worker(client, task_queue="hello", workflows=[HelloWorkflow],
activities=[greet])
    await worker.run()                # polls forever

if __name__ == "__main__":
    asyncio.run(main())
```

PYTHON

```
# starter.py
import asyncio
from temporalio.client import Client
from workflows import HelloWorldWorkflow

async def main():
    client = await Client.connect("localhost:7233")
    result = await client.execute_workflow(
        HelloWorldWorkflow.run, "Ahmed", id="hello-ahmed", task_queue="hello"
    )
    print(result)

asyncio.run(main())
```

SHELL

```
uv run worker.py          # terminal 1
uv run starter.py         # terminal 2 -> Hello, Ahmed!
```

NODE.JS MENTAL MODEL

The TypeScript SDK has the same shape:

TYPESCRIPT

```
const { greet } = proxyActivities<typeof activities>({ startToCloseTimeout: '10
seconds' });
export async function helloWorkflow(name: string): Promise<string> {
    return await greet(name);
}
```

`proxyActivities` gives you a typed stub: calling it schedules the activity on the server instead of running it inline. Python's `workflow.execute_activity(fn, arg)` is the same idea spelled explicitly.

3.2 Who runs what

Piece	Runs on	Can do I/O?	Stored?
Workflow code	Your worker	No	Its history is
Activity code	Your worker	Yes	Only inputs, results, failures
Client	Your API, CLI, cron, anywhere	Yes	No
Temporal server	Temporal cluster or Cloud	—	Histories, timers, queues

The server **never runs your code**. It stores events and hands out tasks. Your worker polls a **task queue**, executes a workflow step or an activity, and reports back. You scale by running more worker processes on the same task queue, exactly like adding BullMQ workers.

3.3 Prove the durability to yourself

Make the workflow long enough to kill:

PYTHON

```
@workflow.defn
class CountWorkflow:
    @workflow.run
    async def run(self, n: int) -> int:
        total = 0
        for i in range(n):
            total += await workflow.execute_activity(
                slow_add, i, start_to_close_timeout=timedelta(seconds=30))
            workflow.logger.info("step %d, total=%d", i, total)
        return total
```

Where `slow_add` is an activity that sleeps 2 seconds and returns `i`. Start it with `n=10`, and at step 4 hit **Ctrl-C** on the worker. Restart the worker. The log resumes at step 5, `total` is correct, and steps 0–4 are never re-executed.

COMMON MISTAKE

Expecting the workflow's `logger.info` lines for steps 0–4 to print again after the restart. During replay, the SDK **suppresses** workflow logs by default, so replay looks quiet. The activities really weren't re-run, and the event history in the UI proves it.

3.4 Data in and out

Workflow and activity arguments and results are **serialized** (JSON by default) and stored in history. Use dataclasses so types survive the round trip:

PYTHON

```
from dataclasses import dataclass

@dataclass
class Order:
    id: str
    amount_cents: int
    email: str
```

Keep payloads **small**. There is a size limit per payload (about 2 MB) and every byte is stored in history. Pass an S3 key or a database ID, not the file or the whole row set.

TIP

Prefer **one dataclass argument** over several positional ones. Adding a field with a default later is backwards compatible. Adding a new positional argument breaks workflows that are already running.

EXERCISE

Build `CountWorkflow`, kill the worker mid-run, restart it, and find in the UI's event history the exact event where the restarted worker picked up.

Part 4 · Activities: Where Side Effects Live

4.1 What belongs in an activity

Anything that touches the outside world: HTTP calls, database reads and writes, sending email, calling an LLM, reading files, the current time for business logic, random IDs you store. If it can fail, be slow, or return something different next time, it's an activity.

4.2 Timeouts: always set one

Timeout	Measures	Use it for
<code>start_to_close_timeout</code>	One attempt, from start to finish	Always set this. Max time one try may take
<code>schedule_to_close_timeout</code>	All attempts together, including retries	Hard deadline for the whole activity
<code>schedule_to_start_timeout</code>	Waiting in the queue for a worker	Detect "no workers running" (rarely needed)
<code>heartbeat_timeout</code>	Gap between heartbeats	Detect a dead worker on long activities fast

PYTHON

```
await workflow.execute_activity(
    fetch_invoice, invoice_id,
    start_to_close_timeout=timedelta(seconds=20),      # each try
    schedule_to_close_timeout=timedelta(minutes=10),  # give up entirely after 10 min
)
```

COMMON MISTAKE

Setting only `schedule_to_close_timeout` to "1 day" for an HTTP call. If the worker dies mid-attempt, Temporal can't tell. It waits for the attempt to time out, and without a `start_to_close_timeout` that means waiting until the whole day has passed. Always bound each attempt.

4.3 Retries are on by default

Activities retry **forever** with exponential backoff unless you say otherwise. Tune it with a `RetryPolicy`:

PYTHON

```

from temporalio.common import RetryPolicy

await workflow.execute_activity(
    charge_card, order,
    start_to_close_timeout=timedelta(seconds=30),
    retry_policy=RetryPolicy(
        initial_interval=timedelta(seconds=1),
        backoff_coefficient=2.0,           # 1s, 2s, 4s, 8s ...
        maximum_interval=timedelta(minutes=1),
        maximum_attempts=5,              # 0 = unlimited
        non_retryable_error_types=["CardDeclined"],
    ),
)

```

Mark business failures as non-retryable **from inside the activity**, since retrying a declined card is pointless:

PYTHON

```

from temporalio.exceptions import ApplicationError

@activity.defn
async def charge_card(order: Order) -> str:
    resp = await stripe_charge(order)
    if resp.status == "declined":
        raise ApplicationError("card declined", type="CardDeclined", non_retryable=True)
    return resp.charge_id

```

NODE.JS MENTAL MODEL

BullMQ equivalent: { attempts: 5, backoff: { type: 'exponential', delay: 1000 } } plus throw new UnrecoverableError() for failures you shouldn't retry. Temporal's version is per call site, and the retry state survives worker restarts.

4.4 Idempotency: the rule that makes retries safe

An activity can run **more than once**. A worker may finish the charge, then crash before reporting success, and Temporal retries it. So every side-effecting activity must be idempotent:

PYTHON

```

@activity.defn
async def charge_card(order: Order) -> str:
    info = activity.info()
    key = f"{info.workflow_id}-charge"           # stable across retries
    return await payments.charge(amount=order.amount_cents, idempotency_key=key) # your
payment client

```

Use the workflow ID (your business key) plus a step name as the idempotency key for payments, emails and inserts. For database writes, prefer `INSERT ... ON CONFLICT DO NOTHING` or upserts.

COMMON MISTAKE

Using `activity.info().attempt` or a fresh `uuid4()` inside the activity as the idempotency key. Both change on every retry, which defeats the point. The key must be the same for every attempt of the same logical step.

4.5 Long activities: heartbeats

For activities that take minutes (a big export, a video encode, a batch of LLM calls), **heartbeat**. It tells Temporal you're alive, lets you resume from progress after a crash, and delivers cancellation.

PYTHON

```
@activity.defn
async def export_rows(job: ExportJob) -> int:
    start = activity.info().heartbeat_details[0] if activity.info().heartbeat_details else 0
    for i in range(start, job.total, 500):
        await write_batch(job, i, 500)
        activity.heartbeat(i + 500)          # progress survives a crash
    return job.total
```

PYTHON

```
await workflow.execute_activity(
    export_rows, job,
    start_to_close_timeout=timedelta(hours=2),
    heartbeat_timeout=timedelta(seconds=30),    # no heartbeat for 30s -> retry elsewhere
)
```

4.6 Sync activities and blocking libraries

`async def` activities run on the worker's event loop. Blocking calls there (such as `requests`, or a sync DB driver) freeze **every** activity on that worker. Either use async libraries (`httpx`, `asyncpg`) or write a plain `def` activity and give the worker a thread pool:

PYTHON

```
from concurrent.futures import ThreadPoolExecutor

@activity.defn
def resize_image(path: str) -> str:          # sync: runs in the thread pool
    ...

worker = Worker(client, task_queue="media", activities=[resize_image],
                activity_executor=ThreadPoolExecutor(max_workers=10))
```

WHEN NOT TO USE IT

Don't split one small, fast, idempotent operation into five activities "for durability". Every activity adds a round trip and history events. The right size for an activity is **one retryable unit of work**: one API call, one transaction, one batch.

EXERCISE

Write a `flaky_fetch` activity that raises on its first two attempts (check `activity.info().attempt`) and succeeds on the third. Watch the retries in the UI's pending activity view. Then make it raise a non-retryable `ApplicationError` and watch the workflow fail immediately.

Part 5 · Workflow Rules: Determinism

5.1 Why the rules exist

Replay (Part 1.2) re-runs your workflow function from the top and checks each command against the recorded history. On the first run the workflow scheduled `charge`, then `reserve_stock`. On replay it **must** do exactly that again, in the same order. If the code takes a different branch, Temporal can't match it to history and raises a **non-determinism error**. The workflow gets stuck until you fix the code.

So workflow code must produce the **same commands given the same history**. Everything nondeterministic goes in activities, or goes through the SDK's deterministic replacements.

5.2 The rules, as a table

Don't do this in a workflow	Why	Do this instead
<code>datetime.now()</code> / <code>time.time()</code>	Different on replay	<code>workflow.now()</code>
<code>random.random()</code> , <code>uuid.uuid4()</code>	Different on replay	<code>workflow.random()</code> , <code>workflow.uuid4()</code>
HTTP, DB, file, env-var reads	Side effect, may change	An activity
<code>time.sleep()</code>	Blocks the worker	<code>await asyncio.sleep(...)</code> (a durable timer)
Threads, <code>multiprocessing</code>	Nondeterministic scheduling	<code>asyncio.gather</code> of activities
Iterating a <code>set</code> for ordering	Order can vary	Sort, or use a <code>list</code> or <code>dict</code>
Reading mutable global state	Differs between workers	Pass it in as input

PYTHON

```
@workflow.defn
class Trial:
    @workflow.run
    async def run(self, user_id: str) -> None:
        started = workflow.now()                # recorded, identical on replay
        coupon = str(workflow.uuid4())           # deterministic UUID
        await asyncio.sleep(timedelta(days=14).total_seconds())
        await workflow.execute_activity(
            end_trial, EndTrial(user_id, started.isoformat(), coupon),
            start_to_close_timeout=timedelta(seconds=30))
```

NODE.JS MENTAL MODEL

It's the same discipline as a Redux reducer or a React render: given the same inputs, produce the same output with no side effects. The side effects go in the "thunk" or "effect", and in Temporal that's the activity.

5.3 The Python sandbox

The Python SDK runs workflows in a **sandbox** that re-imports your workflow module per run and blocks many known-nondeterministic calls. If you accidentally call `datetime.now()` or open a file in workflow code, you'll often get a clear `RestrictedWorkflowAccessError` at test time instead of a subtle bug in production.

Two habits keep the sandbox happy:

PYTHON

```
from temporalio import workflow

with workflow.unsafe.imports_passed_through():
    import pydantic                # heavy, side-effect-free libs: pass through
    from activities import charge, reserve_stock
```

Pass through imports of third-party modules and your activity module, so the sandbox doesn't re-import them on every workflow run. Keep the workflow module itself thin: dataclasses, workflow classes and not much else.

COMMON MISTAKE

Importing `httpx`, `openai` or your ORM at the top of `workflows.py` without passing it through. It works in a small demo, then makes workers slow and memory-hungry, or trips the sandbox on import side effects. Heavy clients belong in `activities.py`.

5.4 Logging and printing

Use `workflow.logger`. It adds workflow context (ID, run ID) and **stays quiet during replay**, so you don't get duplicated lines when a worker restarts. A plain `print()` prints again on every replay.

5.5 Changing code of running workflows

Determinism also constrains **deploys**. If ten thousand workflows are sleeping inside `ProcessOrder` and you insert a new activity between two old ones, replaying their histories against the new code fails. Part 10 shows `workflow.patched()` for exactly this. For now, the rule is: **adding code after the last `await` a running workflow has reached is safe, and reordering or inserting earlier steps isn't.**

TIP

Run a **replay test** (Part 9.4) in CI against real histories from production. It catches non-determinism before a deploy, not after.

EXERCISE

In a workflow, call `datetime.now()` and run it. Read the sandbox error. Then replace it with `workflow.now()`. Next, write a workflow that branches on `workflow.random().random() < 0.5`, run it several times, and confirm in the UI that each run's replay is stable.

Part 6 · Talking to Running Workflows

A workflow can run for weeks, and the outside world needs to talk to it: "the user approved", "what's the status?", "change the delivery address". Temporal has three message types.

Message	Direction	Can change state?	Returns a value?	Use for
Signal	In, async	Yes	No	Events: approved, paid, cancelled
Query	Out, read-only	No	Yes	Status pages, progress bars
Update	In, sync	Yes	Yes (and can be validated)	"Change X and tell me the result"

6.1 An approval workflow

PYTHON

```

@workflow.defn
class ExpenseApproval:
    def __init__(self) -> None:
        self.decision: str | None = None
        self.status = "waiting"

    @workflow.run
    async def run(self, expense: Expense) -> str:
        await workflow.execute_activity(notify_manager, expense,
                                       start_to_close_timeout=timedelta(seconds=30))

        try:
            await workflow.wait_condition(lambda: self.decision is not None,
                                         timeout=timedelta(days=3))

        except asyncio.TimeoutError:
            self.decision = "auto-rejected"
        self.status = self.decision
        if self.decision == "approved":
            await workflow.execute_activity(pay_out, expense,
                                           start_to_close_timeout=timedelta(seconds=30))

        return self.decision

    @workflow.signal
    async def decide(self, decision: str) -> None:
        self.decision = decision

    @workflow.query
    def current_status(self) -> str:
        return self.status

```

`wait_condition` blocks durably until the lambda is true: no polling, and no worker is tied up while it waits. With a timeout, you get the classic "approve within 3 days or auto-reject" in five lines.

From your API (FastAPI, Express via the TS client, anything):

PYTHON

```
handle = client.get_workflow_handle("expense-42")
await handle.signal(ExpenseApproval.decide, "approved")
status = await handle.query(ExpenseApproval.current_status)
```

NODE.JS MENTAL MODEL

In n8n this is a **Wait** node set to "resume on webhook call". In a hand-rolled Node system it's a `status` column, a webhook handler that updates it, and a sweeper that auto-rejects after 3 days. Here it's one class.

6.2 Updates: a signal that answers back

A signal is fire-and-forget: the caller can't know if it was accepted. An **update** runs a handler and returns its result (or error) to the caller, and can **validate** before anything is written to history:

PYTHON

```
@workflow.defn
class Cart:
    def __init__(self) -> None:
        self.items: list[str] = []

    @workflow.run
    async def run(self) -> list[str]:
        await workflow.wait_condition(lambda: len(self.items) >= 10)
        return self.items

    @workflow.update
    async def add_item(self, sku: str) -> int:
        self.items.append(sku)
        return len(self.items)

    @add_item.validator
    def validate_add(self, sku: str) -> None:
        if not sku.startswith("SKU-"):
            raise ValueError("bad sku")           # rejected: nothing recorded
```

PYTHON

```
count = await handle.execute_update(Cart.add_item, "SKU-123")
```

TIP

Choose an **update** when the caller needs to know the outcome ("was my item added?"), a **signal** for events where the caller doesn't wait, and a **query** for reads. Queries must never change state, and they run even on a completed workflow.

6.3 Timers

In workflow code, `await asyncio.sleep(seconds)` is a **durable timer** stored on the server, so sleeping 30 days costs nothing. Combine timers and signals for "remind, then escalate":

PYTHON

```
for reminder in range(3):
    if await self._wait_for_decision(timedelta(days=1)):
        break
    await workflow.execute_activity(send_reminder, reminder, start_to_close_timeout=timedelta(seconds=30))
```

where `_wait_for_decision` wraps `wait_condition` with a timeout and returns `True` or `False`.

6.4 Cancellation

`handle.cancel()` asks the workflow to stop. It arrives as `asyncio.CancelledError` at the current `await`, so you can clean up:

PYTHON

```
try:
    await workflow.execute_activity(long_job, ..., start_to_close_timeout=timedelta(hours=1),
                                   heartbeat_timeout=timedelta(seconds=30))
except asyncio.CancelledError:
    await workflow.execute_activity(cleanup, ..., start_to_close_timeout=timedelta(seconds=30))
    raise
```

`terminate` skips all of that: it's the kill -9 for workflows.

COMMON MISTAKE

Doing I/O inside a signal or update handler. Handlers are workflow code, so the same determinism rules apply. If a handler needs I/O, call `workflow.execute_activity` from the handler (async handlers may await activities), or set state and let the main `run` loop do it.

EXERCISE

Build `ExpenseApproval` . Start it, then query its status, signal `approved` from a tiny FastAPI endpoint, and query again. Run a second one and let the 3-day timeout fire in seconds, using the time-skipping test environment from Part 9.

Part 7 · Composition: Parallelism, Children, Sagas

7.1 Run activities in parallel

Workflow code is `asyncio`, so fan-out is `asyncio.gather`. Temporal's event loop keeps it deterministic:

PYTHON

```
@workflow.run
async def run(self, urls: list[str]) -> list[Summary]:
    opts = dict(start_to_close_timeout=timedelta(minutes=2))
    pages = await asyncio.gather(*(workflow.execute_activity(fetch_page, u, **opts) for u
in urls))
    return await asyncio.gather(*(workflow.execute_activity(summarize, p, **opts) for p in p
ages))
```

Each activity retries on its own. If page 7 fails twice then succeeds, pages 1-6 aren't refetched.

NODE.JS MENTAL MODEL

It's `Promise.all(urls.map(fetchPage))`, but each promise is individually durable and retried. `Promise.allSettled` maps to `asyncio.gather(..., return_exceptions=True)`.

TIP

Fanning out to 10,000 activities in one workflow bloats its history. Batch them (one activity processes 100 items), or split the work across **child workflows**, each handling a slice.

7.2 Child workflows

A child workflow has its own ID, history and retry behaviour. Use them to split a huge process into parts, to reuse a workflow as a building block, or to keep each history small.

PYTHON

```

@workflow.run
async def run(self, batch: Batch) -> int:
    results = await asyncio.gather(*(
        workflow.execute_child_workflow(ProcessCustomer.run, c, id=f"{workflow.info().workfl
ow_id}-{c.id}")
        for c in batch.customers
    ))
    return sum(results)

```

Choose	When
Activity	One unit of I/O work
Child workflow	A multi-step process with its own lifecycle, retries or signals
Separate top-level workflow	Independent lifecycle, started by the client

7.3 Continue-as-new: workflows that run forever

A subscription or a polling loop never ends, but a single history can't grow forever (Temporal warns at thousands of events and has a hard limit around 50,000). The fix: at a checkpoint, finish this run and **start a fresh run with the same workflow ID**, carrying the state forward.

PYTHON

```

@workflow.defn
class Subscription:
    @workflow.run
    async def run(self, state: SubState) -> None:
        while True:
            await asyncio.sleep(timedelta(days=30).total_seconds())
            await workflow.execute_activity(bill_customer, state,
                                           start_to_close_timeout=timedelta(minutes=1))

            state.cycles += 1
            if workflow.info().get_current_history_length() > 5_000:
                workflow.continue_as_new(state)      # new run, same ID, fresh history

```

7.4 Sagas: undo on failure

Distributed transactions don't exist across Stripe, your inventory service and an airline API. A **saga** records a compensation for each completed step and runs them in reverse if a later step fails:

PYTHON

```

@workflow.run
async def run(self, trip: Trip) -> str:
    compensations = []
    opts = dict(start_to_close_timeout=timedelta(seconds=30))
    try:
        await workflow.execute_activity(book_flight, trip, **opts)
        compensations.append(cancel_flight)
        await workflow.execute_activity(book_hotel, trip, **opts)
        compensations.append(cancel_hotel)
        await workflow.execute_activity(charge_card, trip, **opts)
        compensations.append(refund_card)
        return "booked"
    except Exception:
        for undo in reversed(compensations):
            await workflow.execute_activity(undo, trip, **opts) # retried until it
succeeds
        raise

```

What makes this reliable is that the compensations are **durable too**. If the worker dies halfway through rolling back, the rollback resumes. That's the part that's nearly impossible to get right with queues.

COMMON MISTAKE

Appending the compensation **before** the step succeeds. If `book_hotel` never happened, `cancel_hotel` shouldn't run. Append after the `await` returns. Also make each compensation idempotent and tolerant of "nothing to undo".

EXERCISE

Build the trip saga with a `charge_card` that always raises a non-retryable error. In the event history, confirm that `cancel_hotel` then `cancel_flight` ran, in that order, and that killing the worker mid-rollback doesn't skip either.

Part 8 · Starting Workflows: IDs, Dedup and Schedules

8.1 Start vs execute

PYTHON

```
# wait for the result (fine for short workflows, scripts, tests)
result = await client.execute_workflow(Onboard.run, user, id=f"onboard-{user.id}", task_queue="users")

# start and return immediately with a handle (what your API should do)
handle = await client.start_workflow(Onboard.run, user, id=f"onboard-{user.id}", task_queue="users")
return {"workflow_id": handle.id}
```

An HTTP handler should **start** and return. Later, the client can `get_workflow_handle(id)` to query, signal or await `handle.result()`.

8.2 Workflow IDs are your dedup key

Only one **running** workflow may have a given ID. Use your business key: `order-123`, `onboard-user-9`, `invoice-2026-09`. A double-click, a retried webhook or two API pods racing all collapse into a single workflow.

PYTHON

```
from temporalio.common import WorkflowIDConflictPolicy, WorkflowIDReusePolicy

await client.start_workflow(
    ProcessPayment.run, payment,
    id=f"payment-{payment.id}",
    task_queue="payments",
    id_conflict_policy=WorkflowIDConflictPolicy.USE_EXISTING,      # already running? return
    it
    id_reuse_policy=WorkflowIDReusePolicy.REJECT_DUPLICATE,      # finished before? never
    rerun
)
```

Policy	Question it answers	Typical choice
Conflict policy	A run with this ID is running now. What happens?	<code>USE_EXISTING</code> for idempotent starts, or fail
Reuse policy	A run with this ID finished before. May it run again?	<code>REJECT_DUPLICATE</code> for payments, allow for daily jobs

NODE.JS MENTAL MODEL

It replaces the Redis `SETNX lock:order-123` plus "check the status column first" dance you'd do in Node before enqueueing a BullMQ job. BullMQ's `jobId` option is the closest cousin, but it only dedupes while the job exists.

8.3 Schedules: cron, but durable

Temporal Schedules start a workflow on a calendar or interval, with pause, backfill and overlap control, and they're visible in the UI.

PYTHON

```
from temporalio.client import (Schedule, ScheduleActionStartWorkflow, ScheduleSpec,
                               ScheduleIntervalSpec)

await client.create_schedule(
    "daily-digest",
    Schedule(
        action=ScheduleActionStartWorkflow(DailyDigest.run, "all-users",
                                            id="daily-digest", task_queue="digests"),
        spec=ScheduleSpec(cron_expressions=["0 7 * * *"]),          # 07:00 every day
    ),
)
# or: ScheduleSpec(intervals=[ScheduleIntervalSpec(every=timedelta(hours=1))])
```

SHELL

```
temporal schedule list
temporal schedule toggle --schedule-id daily-digest --pause --reason "holiday"
temporal schedule trigger --schedule-id daily-digest          # run it now
```

TIP

Set the overlap policy deliberately. By default a new run is skipped if the previous one is still running, which is usually what you want for a report. For "every run matters" jobs, buffer them instead. Schedules also support **backfill**: "run for every day last week" after an outage.

WHEN NOT TO USE IT

For a single machine running a single script at 3 a.m., system cron or a Prefect deployment is simpler. Reach for Temporal Schedules when the scheduled thing is itself a durable, multi-step workflow, or when you need pause, backfill and an audit trail.

8.4 Starting from Node

Your Express or Nest API can start Python workflows. Clients and workers can be in different languages, as long as they agree on the **workflow type name**, the **task queue** and the **payload shape**.

TYPESCRIPT

```
import { Client, Connection } from '@temporalio/client';
const client = new Client({ connection: await Connection.connect({ address: 'localhost:
7233' }) });
await client.workflow.start('ProcessOrder', {           // type name as a string
  taskQueue: 'orders', workflowId: `order-${order.id}`, args: [order],
});
```

EXERCISE

Create a schedule that runs a `Heartbeat` workflow every minute. Pause it, trigger it by hand, and backfill the last 10 minutes. Then start the same workflow twice with the same ID and `USE_EXISTING` and confirm that only one run exists.

Part 9 · Testing Workflows

9.1 The time-skipping test environment

The SDK ships a test server that **skips time** whenever all workflows are waiting on timers. A workflow that sleeps 30 days finishes its test in milliseconds.

SHELL

```
uv add --dev pytest pytest-asyncio
```

PYTHON

```
# tests/test_approval.py
import pytest
from temporalio.testing import WorkflowEnvironment
from temporalio.worker import Worker
from workflows import ExpenseApproval
from activities import notify_manager, pay_out

@pytest.mark.asyncio
async def test_auto_reject_after_three_days():
    async with await WorkflowEnvironment.start_time_skipping() as env:
        async with Worker(env.client, task_queue="t", workflows=[ExpenseApproval],
                          activities=[notify_manager, pay_out]):
            result = await env.client.execute_workflow(
                ExpenseApproval.run, Expense("e1", 4200), id="e1", task_queue="t")
            assert result == "auto-rejected" # 3 days passed in ~0s
```

The first run downloads the test server binary, so run it once on a machine with network access.

9.2 Mock activities

Register a fake activity **with the same name** as the real one. Your workflow calls it without knowing:

PYTHON

```
from temporalio import activity

@activity.defn(name="pay_out")
async def pay_out_mock(expense: Expense) -> None:
    calls.append(expense.id) # record instead of paying

@activity.defn(name="notify_manager")
async def notify_mock(expense: Expense) -> None:
    pass
```

Then pass `activities=[notify_mock, pay_out_mock]` to the test worker.

NODE.JS MENTAL MODEL

It's `jest.mock('./activities')`, done at the worker level instead of the module level. Because workflows only reference activities by **name**, swapping implementations is trivial.

9.3 Signals in tests

PYTHON

```
handle = await env.client.start_workflow(ExpenseApproval.run, exp, id="e2", task_queue="t")
await handle.signal(ExpenseApproval.decide, "approved")
assert await handle.result() == "approved"
assert calls == ["e2"]
```

9.4 Replay tests: catch non-determinism before deploy

Download real histories (from production, or from a test run) and replay them against your **new** code. If you changed the workflow in a non-deterministic way, the replay fails.

SHELL

```
temporal workflow show --workflow-id order-123 --output json > histories/order-123.json
```

PYTHON

```
from temporalio.client import WorkflowHistory
from temporalio.worker import Replayer

@pytest.mark.asyncio
async def test_replay_order_123():
    history = WorkflowHistory.from_json("order-123", open("histories/order-123.json").read())
    await Replayer(workflows=[ProcessOrder]).replay_workflow(history) # raises on mismatch
```

TIP

Keep a folder of histories that covers every branch of each workflow, and run the replayer in CI. It's the cheapest insurance you can buy against a broken deploy.

9.5 Test activities as plain functions

Activities are normal functions. For ones that use `activity.info()` or heartbeats, `ActivityEnvironment` supplies the context:

PYTHON

```
from temporalio.testing import ActivityEnvironment

async def test_export_heartbeats():
    env = ActivityEnvironment()
    beats = []
    env.on_heartbeat = lambda *d: beats.append(d)
    assert await env.run(export_rows, ExportJob(total=1000)) == 1000
    assert beats
```

EXERCISE

Write three tests for `ExpenseApproval`: approved (signal), rejected (signal) and auto-rejected (timeout). Then add a replay test from a history you export with the CLI, change the workflow to insert a new activity before `notify_manager`, and watch the replay test fail.

Part 10 · Versioning and Production

10.1 Changing a workflow that's already running

You have 5,000 `ProcessOrder` runs sleeping in their 7-day wait, and you need to add a fraud check after `charge`. Inserting it directly breaks replay for all 5,000 (Part 5.5). **Patching** lets old runs take the old path and new runs take the new one:

PYTHON

```
await workflow.execute_activity(charge, order, **opts)
if workflow.patched("fraud-check-v1"):
    await workflow.execute_activity(fraud_check, order, **opts)  # only new runs
await workflow.execute_activity(reserve_stock, order, **opts)
```

The lifecycle is three deploys:

1. **Patch:** ship the `if workflow.patched(...)` branch.
2. **Deprecate:** once no run started before step 1 is still open, replace the `if` with `workflow.deprecate_patch("fraud-check-v1")` and keep only the new code.
3. **Remove:** once no run from step 1 is still open either, delete the `deprecate_patch` line.

TIP

For big changes, a **new workflow type** (`ProcessOrderV2`) is often simpler than stacking patches. Route new starts to V2 and let V1 drain. Temporal's newer **Worker Versioning** feature can pin runs to the worker build that started them; check the current docs, since this area has evolved.

10.2 Workers in production

- **Stateless and horizontal.** Run N identical worker containers per task queue. Scale on queue backlog (visible in the UI and metrics).
- **Separate task queues by resource.** CPU-heavy activities (`media`), rate-limited APIs (`openai`) and workflows (`orders`) on their own queues and worker pools, so one can't starve another.
- **Graceful shutdown.** On SIGTERM, stop polling and let in-flight activities finish (`graceful_shutdown_timeout` on `Worker`). Kubernetes `terminationGracePeriodSeconds` must be longer.
- **Limit concurrency.** `max_concurrent_activities` on the worker protects downstream APIs, like BullMQ's `concurrency` option.

PYTHON

```
worker = Worker(client, task_queue="openai", activities=[call_llm],
                max_concurrent_activities=8,
                graceful_shutdown_timeout=timedelta(seconds=60))
```

10.3 Data: types and privacy

Use dataclasses or Pydantic models for every input and output. For Pydantic, switch the data converter:

PYTHON

```
from temporalio.contrib.pydantic import pydantic_data_converter
client = await Client.connect("localhost:7233", data_converter=pydantic_data_converter)
```

Everything you pass is stored in history **in plain text** by default. For PII, add a **payload codec** that encrypts payloads before they leave your worker, so the server (and Temporal Cloud) only sees ciphertext. Or store sensitive data elsewhere and pass references.

10.4 Self-host vs Temporal Cloud

	Self-hosted	Temporal Cloud
You run	Server + DB (Postgres, MySQL or Cassandra) + optional Elasticsearch	Only your workers
Cost	Infra + your ops time	Usage-based pricing
Good for	Learning, strict data residency, big platform teams	Most production teams
Gotchas	Upgrades, DB tuning, history shards set at creation	Namespace and mTLS or API-key setup

COMMON MISTAKE

Treating the self-hosted cluster like a stateless app. Its database **is** every running business process. Back it up, monitor it, and plan upgrades with the same care as your main production database.

10.5 Observability

- **UI and CLI** for single runs: history, pending activities, stack trace (the UI can show where a workflow is currently blocked).
- **Metrics:** the SDK emits Prometheus metrics (workflow and activity latencies, failures, schedule-to-start latency). Schedule-to-start latency climbing means you need more workers.

- **Tracing:** OpenTelemetry interceptors (`temporalio.contrib.opentelemetry`) link a workflow's activities into one trace.
- **Search attributes:** index custom fields (`CustomerId` , `OrderTotal`) so you can run `temporal workflow list --query 'CustomerId="c-42"'`.

WHEN NOT TO USE IT

Don't reach for Temporal Cloud or a production cluster while you're learning. The dev server and SQLite file from Part 2 are enough for this whole book.

EXERCISE

Take `ProcessOrder` , start three runs that sleep, then add a patched `fraud_check` . Restart the worker, confirm the old runs finish without it and new runs include it, and add both histories to your replay tests.

Part 11 · AI Workflows on Temporal

LLM work is a natural fit for durable execution. Calls are slow, rate-limited, flaky and **expensive**. When step 9 of a 10-step chain fails, re-running steps 1-8 wastes money. With Temporal, completed LLM calls are in history and are never paid for twice.

11.1 The LLM call as an activity

PYTHON

```
# activities.py
from openai import AsyncOpenAI, RateLimitError
from temporalio import activity
from temporalio.exceptions import ApplicationError

llm = AsyncOpenAI()

@activity.defn
async def complete(req: LLMRequest) -> str:
    try:
        resp = await llm.chat.completions.create(model=req.model, messages=req.messages)
    except RateLimitError:
        raise # retryable: backoff handles 429s
    except Exception as e:
        if "context_length" in str(e):
            raise ApplicationError(str(e), type="ContextTooLong", non_retryable=True)
        raise
    return resp.choices[0].message.content
```

Put LLM activities on their own task queue with a worker-level `max_concurrent_activities`. That's your global concurrency cap for the provider.

NODE.JS MENTAL MODEL

It's the same pattern as the LLM chapter in the Python book, plus retries you don't hand-roll: no `p-retry`, no custom 429 loop. The retry policy is declared at the call site, and backoff survives restarts.

11.2 A document pipeline

PYTHON

```
@workflow.defn
class SummarizeReport:
    @workflow.run
    async def run(self, doc_id: str) -> str:
        opts = dict(start_to_close_timeout=timedelta(minutes=2),
                    retry_policy=RetryPolicy(maximum_interval=timedelta(minutes=1)))
        chunks = await workflow.execute_activity(load_and_chunk, doc_id, **opts)
        partials = await asyncio.gather(*(
            workflow.execute_activity(complete, summarize_prompt(c), task_queue="llm", **opts)
            for c in chunks))
        final = await workflow.execute_activity(complete, merge_prompt(partials),
        task_queue="llm", **opts)
        await workflow.execute_activity(save_summary, SaveReq(doc_id, final), **opts)
        return final
```

Note that `summarize_prompt(c)` runs **in the workflow**. That's fine, because building a prompt from a string is deterministic. Calling the model isn't, so that's the activity.

11.3 A durable agent loop

An agent is a loop: the model picks a tool, you run it, you feed back the result, and repeat. As a workflow, every model call and every tool call is a recorded, retryable step, and a human can intervene with a signal.

PYTHON

```
@workflow.defn
class Agent:
    def __init__(self) -> None:
        self.approved: bool | None = None

    @workflow.run
    async def run(self, goal: str) -> str:
        messages = [system_prompt(), {"role": "user", "content": goal}]
        for step in range(15): # hard step budget
            action = await workflow.execute_activity(
                choose_action, messages, task_queue="llm",
                start_to_close_timeout=timedelta(minutes=2))
            if action.kind == "final":
                return action.text
            if action.tool in RISKY_TOOLS: # human in the loop
                self.approved = None
                await workflow.execute_activity(ask_human, action, start_to_close_timeout=timedelta(seconds=30))
                await workflow.wait_condition(lambda: self.approved is not None, timeout=timedelta(hours=24))
                if not self.approved:
                    messages.append(tool_result(action, "denied by human"))
                    continue
            result = await workflow.execute_activity(
```

```

        run_tool, action, start_to_close_timeout=timedelta(minutes=5))
        messages.append(tool_result(action, result))
    return "step budget exhausted"

@workflow.signal
def approve(self, ok: bool) -> None:
    self.approved = ok

```

The agent can wait a day for approval without holding a process open, survive deploys mid-task, and show a full audit trail of every model decision and tool call in the UI.

TIP

Temporal publishes integrations for popular agent SDKs (for example, a contrib module for the OpenAI Agents SDK) that make each model and tool call an activity automatically. Check the SDK's `contrib` packages before hand-writing the loop for a large project. Writing it once by hand, as above, is still the best way to understand what those integrations do.

COMMON MISTAKE

Putting the whole agent run inside **one** activity. It works until the 14th tool call fails, and then the retry starts the agent from scratch: new model calls, new costs and possibly repeated side effects. Make each model call and tool call its own activity.

WHEN NOT TO USE IT

For a single prompt and response in a request handler, Temporal adds latency and moving parts for nothing. Use it when the AI work is **multi-step, long-running, expensive to redo, or needs human approval**.

EXERCISE

Build `SummarizeReport` with a fake `complete` activity that fails randomly 30% of the time. Run it on a 20-chunk document and count in the history how many completions were retried, and confirm that no successful chunk was summarized twice.

Part 12 · Capstone: Build Your Own mini-n8n

You asked the real question at the start of this track: *could I build something like n8n myself?* With Temporal, the answer is yes, and the durable part is free. n8n's hard problems (retries, waiting, resuming after a crash, human approval) are exactly what Temporal already solves. What's left is an **interpreter**: read a JSON graph of nodes and run it.

12.1 The architecture

n8n concept	mini-n8n on Temporal
Workflow JSON (nodes + connections)	<code>Graph</code> Pydantic model loaded from <code>flows/*.json</code>
A node (HTTP Request, Set, OpenAI...)	A plain async function in a <code>REGISTRY</code> dict
Running a node	One activity, <code>run_node</code> , which dispatches by type
IF, Wait, "Wait for approval"	Logic inside the workflow: pure code, durable timer, signal
Expressions <code>{{ \$json.email }}</code>	<code>{{ enrich.email }}</code> resolved by a tiny <code>render()</code>
Webhook trigger	FastAPI endpoint that starts a workflow
Executions list	The Temporal Web UI
Retry on fail	<code>RetryPolicy(maximum_attempts=node.retries)</code>

The whole engine is about 150 lines, and every piece below was run end-to-end against a real Temporal dev server.

12.2 The definition format

PYTHON

```
# models.py – the workflow definition format (think: an exported n8n workflow JSON)
from typing import Any
from pydantic import BaseModel

class Node(BaseModel):
    id: str
    type: str # "http" | "transform" | "llm" | "if" | "wait" |
    "approval" | "log"
    params: dict[str, Any] = {}
    next: list[str] = [] # for "if": [then_node, else_node]
    retries: int = 3

class Graph(BaseModel):
    name: str
    start: str
    nodes: dict[str, Node]

class NodeCall(BaseModel):
    node_id: str
    type: str
    params: dict[str, Any]
    ctx: dict[str, Any] # outputs so far: {"trigger": {...}, "fetch": {...}}
```

12.3 Nodes are plain functions

PYTHON

```
# nodes.py – every "n8n node" is a plain async function; one activity dispatches to them
import re
from typing import Any, Awaitable, Callable

import httpx
from temporalio import activity
from temporalio.exceptions import ApplicationError

from models import NodeCall

TEMPLATE = re.compile(r"\{\{s*([\w.]+)\s*\}\}")

def lookup(ctx: dict, path: str) -> Any:
    cur: Any = ctx
    for part in path.split("."):
        cur = cur[int(part)] if isinstance(cur, list) else cur[part]
    return cur

def render(value: Any, ctx: dict) -> Any:
    """Resolve {{ fetch.body.title }} expressions, n8n-style, recursively."""
```

```

if isinstance(value, str):
    whole = TEMPLATE.fullmatch(value.strip())
    if whole:
        return lookup(ctx, whole.group(1))
    return TEMPLATE.sub(lambda m: str(lookup(ctx, m.group(1))), value)
if isinstance(value, dict):
    return {k: render(v, ctx) for k, v in value.items()}
if isinstance(value, list):
    return [render(v, ctx) for v in value]
return value

async def http_node(p: dict) -> dict:
    async with httpx.AsyncClient(timeout=20) as http:
        r = await http.request(p.get("method", "GET"), p["url"], json=p.get("json"))
    if r.status_code >= 500 or r.status_code == 429:
        raise RuntimeError(f"retryable HTTP {r.status_code}")
    if r.status_code >= 400:
        raise ApplicationError(f"HTTP {r.status_code}", non_retryable=True)
    body = r.json() if "json" in r.headers.get("content-type", "") else r.text
    return {"status": r.status_code, "body": body}

async def transform_node(p: dict) -> dict:
    return p.get("set", {})

async def log_node(p: dict) -> dict:
    activity.logger.info("log node: %s", p.get("message"))
    return {"logged": p.get("message")}

async def llm_node(p: dict) -> dict:
    # swap in your provider; kept dependency-free here so the capstone runs offline
    return {"text": f"[summary of] {str(p.get('prompt'))[:80]}"}

REGISTRY: dict[str, Callable[[dict], Awaitable[dict]]] = {
    "http": http_node, "transform": transform_node, "log": log_node, "llm": llm_node,
}

@activity.defn
async def run_node(call: NodeCall) -> dict:
    handler = REGISTRY.get(call.type)
    if handler is None:
        raise ApplicationError(f"unknown node type {call.type}", non_retryable=True)
    return await handler(render(call.params, call.ctx))

```

NODE.JS MENTAL MODEL

This is how n8n itself is shaped: each node type is a class with an `execute()` method, and the engine looks nodes up by type name. Adding a "Slack" node here means writing one async function and adding it to `REGISTRY`, the same as writing a custom n8n node, minus the packaging.

12.4 The durable interpreter

PYTHON

```
# engine.py – the durable interpreter: walks the graph, one activity per node
import asyncio
from datetime import timedelta
from typing import Any

from temporalio import workflow
from temporalio.common import RetryPolicy

with workflow.unsafe.imports_passed_through():
    from models import Graph, Node, NodeCall
    from nodes import render, run_node

CONTROL = {"if", "wait", "approval"} # handled in the workflow itself

@workflow.defn
class RunGraph:
    def __init__(self) -> None:
        self.ctx: dict[str, Any] = {}
        self.current: list[str] = []
        self.approvals: dict[str, bool] = {}

    @workflow.run
    async def run(self, graph: Graph, trigger: dict) -> dict:
        self.ctx = {"trigger": trigger}
        frontier = [graph.start]
        while frontier:
            self.current = frontier
            done = await asyncio.gather(*(self._step(graph.nodes[n]) for n in frontier))
            frontier = list(dict.fromkeys(n for n, _ in done)) # dedupe, keep order

        self.current = []
        return self.ctx

    async def _step(self, node: Node) -> list[str]:
        if node.type == "if": # pure logic: deterministic, no activity
            p = render(node.params, self.ctx)
            ok = OPS[p.get("op", "eq"])(p["left"], p["right"])
            self.ctx[node.id] = {"passed": ok}
            return node.next[:1] if ok else node.next[1:2]

        if node.type == "wait": # durable timer: seconds, days, whatever
            await asyncio.sleep(float(node.params["seconds"]))
            self.ctx[node.id] = {"waited": node.params["seconds"]}
            return node.next

        if node.type == "approval": # human in the loop via signal
            await workflow.wait_condition(lambda: node.id in self.approvals)
            self.ctx[node.id] = {"approved": self.approvals[node.id]}
            return node.next if self.approvals[node.id] else []

        self.ctx[node.id] = await workflow.execute_activity(
            run_node, NodeCall(node_id=node.id, type=node.type, params=node.params,
            ctx=self.ctx),
            start_to_close_timeout=timedelta(minutes=2),
```

```

        retry_policy=RetryPolicy(maximum_attempts=node.retries),
    )
    return node.next

@workflow.signal
def approve(self, node_id: str, ok: bool = True) -> None:
    self.approvals[node_id] = ok

@workflow.query
def status(self) -> dict:
    return {"running": self.current, "done": [k for k in self.ctx if k != "trigger"]}

OPS = {"eq": lambda a, b: a == b, "ne": lambda a, b: a != b,
       "gt": lambda a, b: a > b, "lt": lambda a, b: a < b,
       "contains": lambda a, b: b in a}

```

Three details make this correct:

- **Control nodes live in the workflow.** `if` is pure logic, so it's deterministic. `wait` is `asyncio.sleep`, a durable timer. `approval` is a signal plus `wait_condition`. None of them need an activity.
- **Everything with I/O is one activity per node**, each with its own retry policy, so a failed HTTP node retries alone while earlier node outputs stay in history.
- **Parallel branches** run with `asyncio.gather`, one "frontier" of nodes at a time, like n8n executing the next layer of connections.

12.5 Worker and webhook trigger

PYTHON

```

# worker.py
import asyncio
from temporalio.client import Client
from temporalio.contrib.pydantic import pydantic_data_converter
from temporalio.worker import Worker
from engine import RunGraph
from nodes import run_node

async def main() -> None:
    client = await Client.connect("localhost:7233", data_converter=pydantic_data_converter)
    await Worker(client, task_queue="mini-n8n", workflows=[RunGraph],
activities=[run_node]).run()

if __name__ == "__main__":
    asyncio.run(main())

```

PYTHON

```

# api.py – the "Webhook trigger": POST /hooks/<name> starts a durable run
import json
import uuid
from pathlib import Path

```

```

from fastapi import FastAPI, HTTPException
from temporalio.client import Client
from temporalio.contrib.pydantic import pydantic_data_converter

from engine import RunGraph
from models import Graph

app = FastAPI()
FLOWS = Path("flows")

async def client() -> Client:
    if not hasattr(app.state, "temporal"):
        app.state.temporal = await Client.connect("localhost:7233", data_converter=pydantic_data_converter)
    return app.state.temporal

@app.post("/hooks/{name}")
async def trigger(name: str, payload: dict):
    path = FLOWS / f"{name}.json"
    if not path.exists():
        raise HTTPException(404, "no such flow")
    graph = Graph.model_validate(json.loads(path.read_text()))
    handle = await (await client()).start_workflow(
        RunGraph.run, args=[graph, payload],
        id=f"{name}-{payload.get('id', uuid.uuid4().hex[:8])}", task_queue="mini-n8n")
    return {"run_id": handle.id}

@app.get("/runs/{run_id}")
async def status(run_id: str):
    return await (await client()).get_workflow_handle(run_id).query(RunGraph.status)

@app.post("/runs/{run_id}/approve/{node_id}")
async def approve(run_id: str, node_id: str, ok: bool = True):
    await (await client()).get_workflow_handle(run_id).signal(RunGraph.approve, args=[node_id, ok])
    return {"ok": True}

```

12.6 A flow to run

JSON

```
{
  "name": "new-signup",
  "start": "enrich",
  "nodes": {
    "enrich": {
      "id": "enrich",
      "type": "transform",
      "params": {
        "set": {
          "email": "{{ trigger.email }}"
        },
        "plan": "{{ trigger.plan }}"
      },
      "next": ["is_paid"]
    },
    "is_paid": {
      "id": "is_paid",
      "type": "if",
      "params": {
        "left": "{{ enrich.plan }}",
        "op": "eq",
        "right": "pro"
      },
      "next": ["welcome", "nurture"]
    },
    "welcome": {
      "id": "welcome",
      "type": "llm",
      "params": {
        "prompt": "Write a warm welcome for {{ enrich.email }}"
      },
      "next": ["review"]
    },
    "review": {
      "id": "review",
      "type": "approval",
      "next": ["send"]
    },
    "send": {
      "id": "send",
      "type": "log",
      "params": {
        "message": "SEND: {{ welcome.text }}"
      },
      "next": ["nurture"]
    },
    "nurture": {
      "id": "nurture",
      "type": "wait",
      "params": {
        "seconds": 2
      },
      "next": ["remind"]
    },
    "remind": {
      "id": "remind",
      "type": "log",
      "params": {
        "message": "nurture {{ enrich.email }}"
      }
    }
  }
}
```

SHELL

```
temporal server start-dev           # terminal 1
uv run worker.py                    # terminal 2
uv run uvicorn api:app --port 8000  # terminal 3 (uv add fastapi uvicorn httpx)

curl -X POST localhost:8000/hooks/new-signup -H 'content-type: application/json' \
  -d '{"id": "u1", "email": "sara@example.com", "plan": "pro"}'
curl localhost:8000/runs/new-signup-u1      # {"running": ["review"], ...}
curl -X POST localhost:8000/runs/new-signup-u1/approve/review
```

A **pro** signup is enriched, branches on **is_paid**, gets an LLM welcome, **waits for a human** to approve, then sends. A **free** signup takes the other branch: a durable **wait**, then a nurture log. Kill the worker while a run is waiting, restart it, and approve. The run carries on as if nothing happened.

TIP

What real n8n adds on top is mostly **product**: the visual editor, hundreds of prebuilt nodes, credentials management and the executions UI. The engine is the easy part once you have durable execution. If you want a visual editor, React Flow can produce this same JSON format.

WHEN NOT TO USE IT

Don't build your own engine to ship business automations quickly. Use n8n for that. Build it when automation *is* your product (for example an internal platform where developers register nodes in Python), when you need guarantees n8n can't give, or, as here, to understand deeply how these tools work.

EXERCISE

Extend mini-n8n with: (1) a real `llm` node that calls your provider; (2) a `slack` node using an incoming-webhook URL; (3) a `schedule` trigger that uses a Temporal Schedule to start a flow every morning; (4) a per-node `timeout_seconds` param. Then kill the worker mid-run and prove the run resumes.

Part 13 · Cheat Sheet

13.1 The skeleton

PYTHON

```

from datetime import timedelta
from temporalio import activity, workflow
from temporalio.common import RetryPolicy

@activity.defn
async def do_io(x: str) -> str: ...           # side effects here

@workflow.defn
class MyFlow:
    @workflow.run
    async def run(self, x: str) -> str:
        return await workflow.execute_activity(
            do_io, x, start_to_close_timeout=timedelta(seconds=30),
            retry_policy=RetryPolicy(maximum_attempts=5))

    @workflow.signal
    async def poke(self, v: str) -> None: ...

    @workflow.query
    def state(self) -> str: ...

    @workflow.update
    async def change(self, v: str) -> int: ...

```

13.2 Workflow-safe replacements

Instead of	Use
<code>datetime.now()</code>	<code>workflow.now()</code>
<code>uuid.uuid4()</code> / <code>random</code>	<code>workflow.uuid4()</code> / <code>workflow.random()</code>
<code>time.sleep(n)</code>	<code>await asyncio.sleep(n)</code> (durable timer)
polling loop	<code>await workflow.wait_condition(lambda: ..., timeout=...)</code>
<code>print</code> / <code>logging</code>	<code>workflow.logger</code>
HTTP / DB / files	an activity

13.3 Client calls

Goal	Call
Start, don't wait	<code>await client.start_workflow(Flow.run, arg, id=..., task_queue=...)</code>
Start and wait	<code>await client.execute_workflow(...)</code>
Find a run	<code>client.get_workflow_handle("order-123")</code>
Signal / query / update	<code>handle.signal(...)</code> · <code>handle.query(...)</code> · <code>handle.execute_update(...)</code>
Result / cancel / terminate	<code>handle.result()</code> · <code>handle.cancel()</code> · <code>handle.terminate()</code>
Schedule	<code>client.create_schedule(id, Schedule(action=..., spec=ScheduleSpec(...)))</code>

13.4 Choosing a timeout

Situation	Set
Any activity	<code>start_to_close_timeout</code> (always)
Hard overall deadline	plus <code>schedule_to_close_timeout</code>
Long activity (minutes+)	plus <code>heartbeat_timeout</code> and call <code>activity.heartbeat()</code>
Wait for a human	<code>wait_condition(..., timeout=timedelta(days=N))</code>

13.5 Node.js → Temporal map

In Node you'd use	Temporal gives you
BullMQ job + <code>attempts</code> + <code>backoff</code>	Activity + <code>RetryPolicy</code>
<code>status</code> column + sweeper cron	Workflow state + event history
<code>setTimeout</code> for days (impossible)	<code>asyncio.sleep</code> durable timer
Webhook updates a DB row, poller reacts	Signal / update + <code>wait_condition</code>
Redis <code>SETNX</code> lock per order	Workflow ID + conflict / reuse policies
<code>node-cron</code>	Temporal Schedules
Hand-written rollback code	Saga with durable compensations
TS SDK <code>proxyActivities</code>	Python <code>workflow.execute_activity</code>

13.6 The five rules

1. **Workflows decide, activities do.** No I/O in workflow code.
2. **Every activity is idempotent.** It will run twice one day.
3. **Always set** `start_to_close_timeout`.
4. **Workflow ID = business key.** That's your dedup.

5. **Never change a running workflow's path without `patched()` or a new type**, and prove it with replay tests.

13.7 Temporal vs Prefect vs n8n: pick in 10 seconds

If the job is...	Pick
A business process that must never lose its place (orders, payments, onboarding, agents)	Temporal
A Python data, ML or AI pipeline you schedule and monitor	Prefect
Glue between SaaS apps, built fast, maybe by non-developers	n8n
One background job	A queue (BullMQ, Celery, RQ)